Escalate — Full Build Plan (A to Z)

Oct 28, 2026 · @Zain Ismail

1. What we ship

In 4 weeks, one developer ships Escalate v0.1: a self-hostable AI agent that takes a consumer complaint plus evidence and runs it from first message to consumer court filing, with the user approving every step.

This plan is the build companion to the Software Design & Requirements doc: that one says *what*, this one says *how*, in order.

v0.1 scope

- 3 case types: **refund/replacement refused**, **subscription won't cancel**, **I was scammed**.
- **Global from day one:** any country works through a generic international playbook (company → final notice → national consumer authority or ombudsman → small-claims or consumer court). Starter country packs: Pakistan, India, UK, US; the community adds the rest.
- Drafts in the user's chosen language; UI in English first, with every string translatable.
- Web UI, email from the user's own mailbox, reminders, case timeline, PDF case file.
- **$0 to build and run:** a local open model via Ollama by default; free cloud tiers optional; paid APIs never required.

Definition of done

- [ ] A new user runs `docker compose up` and opens a case in under 5 minutes.
- [ ] The "broken LED, 8 months waiting" demo case runs end to end: intake → seller email → legal notice → court complaint draft → PDF.
- [ ] No message can be sent without an approval click (test proves it).
- [ ] Golden-set evals pass (section 11).
- [ ] README with demo GIF, architecture diagram and contributing guide is live on GitHub with a v0.1.0 release.

Global and free by design

Anyone in any country can run Escalate at zero cost: the default stack is free, open-source software plus a local model, and every paid service is optional.

Global

- `country` is an ISO 3166-1 code and `region` an ISO 3166-2 subdivision (e.g. `US-CA`, `IN-MH`, `PK-SD`); currency and dates are formatted with Babel.
- Playbook resolution falls back step by step: company → region → country → `intl/generic`. Unsupported countries still get a sensible ladder, clearly labelled "generic — check your local rules".
- v0.1 packs: `intl/generic`, `pk`, `in`, `gb`, `us`. Each needs official sources before merge.
- The user writes in any language; drafts come out in the language they pick for the recipient.
- Scam checklist per country (bank fraud line, national reporting body), with a generic fallback: "call the number on the back of your card now".

The $0 path

Component	Free choice	Notes
LLM (default)	Ollama + an open 7–8B model on the user's own machine	Free and private: complaint data never leaves the machine; needs about 8–16 GB RAM
LLM (free cloud fallback)	Groq, Google AI Studio (Gemini Flash), OpenRouter `:free` models	No card needed at time of writing; rate-limited (OpenRouter: 20 requests/min, 50/day); free tiers may use prompts for training, so the UI warns before sending real personal data
OCR	Tesseract + language packs	Free, offline
Email	Gmail SMTP with an app password, or Gmail API	Free
Database, scheduler, PDF	SQLite, APScheduler, WeasyPrint	Free, open source

Component	Free choice	Notes
CI	GitHub Actions	Free for public repos
Hosting	The user's own computer via Docker	Free; no server needed
Public demo	Demo GIF + video in the README; optional free hosting (section 12)	Free

All providers speak the OpenAI-compatible API, so switching between them is one line in `.env`. Free tiers change often; never hard-code one.

2. Prerequisites

You need intermediate Python, a laptop with 8 GB RAM, and four free accounts; total setup takes about 2 hours.

Skills (learn as you go if missing)

- Python 3.12: functions, classes, type hints, async basics
- SQL basics and SQLAlchemy ORM
- Git and GitHub: branches, pull requests, Actions
- Prompt writing and JSON-structured LLM output
- Docker basics: Dockerfile, compose

Accounts and keys

Account	Used for	Cost
GitHub	Repo, Actions CI, releases	Free
Ollama (no account)	Local open model, the default LLM	Free
Groq, Google AI Studio or OpenRouter	Optional free cloud LLM key for weak machines and the demo	Free tier, no card
Gmail	SMTP app password or Gmail API OAuth client	Free

Account	Used for	Cost
Anthropic / OpenAI (optional)	Higher-quality drafting for users who choose to pay	Pay per use; never required

Install on your machine

- Python 3.12, `uv` (package manager), Git, Docker Desktop
- Tesseract OCR plus language packs for the scripts your users write in (`tesseract-ocr`, `tesseract-ocr-urd`)
- VS Code or Cursor with the Python and Ruff extensions

3. Final tech stack

One language (Python), one process for the API and scheduler, one for the UI; pin the latest stable version of each package in `uv.lock` on day 0.

Layer	Choice	Notes
Language	Python 3.12	Type hints everywhere
Package manager	uv	Fast installs, lockfile
Agent orchestration	LangGraph	State graph + SQLite checkpointer for long-running cases
LLM client	`openai` Python SDK pointed at any OpenAI-compatible endpoint	One client for Ollama, Groq, Gemini, OpenRouter and paid APIs
LLM (default, $0)	Ollama + an open 7–8B instruct model	Runs offline; `MODEL_SMART` and `MODEL_FAST` can be the same model
LLM (free cloud)	Groq / Gemini Flash / OpenRouter `:free`	For weak hardware and the demo; rate-limited
LLM (optional paid)	Claude or another paid API	Only if a user wants higher quality

Layer	Choice	Notes
Structured output	Pydantic models + JSON repair and retry	Small local models need the retry
API	FastAPI + Uvicorn	REST, auto docs at `/docs`
Database	SQLite (default) / Postgres via SQLAlchemy 2 + Alembic	Same models both ways
Scheduler	APScheduler with SQLAlchemy job store	Survives restarts
OCR	pytesseract with configurable languages	Vision-model fallback is optional
PDF reading	pypdf	Text from uploaded PDFs
Email	Gmail SMTP (app password) or Gmail API	Sends from the user's own address
Templates	Jinja2 (Markdown)	Letters and legal notices, per country and language
Localisation	Babel + JSON string files	Currency, dates, UI translations
PDF export	WeasyPrint	HTML → case-file PDF
Encryption	`cryptography` (Fernet)	Evidence files at rest
UI	Streamlit	Phase 2: Next.js
Tests	pytest, pytest-asyncio, freezegun	`freezegun` for time travel in scheduler tests
Lint / format	Ruff, mypy	Run in CI
CI/CD	GitHub Actions	Free for public repos
Packaging	Docker + docker compose	One-command install, Ollama included

4. Day 0 setup

By the end of day 0 the repo exists, CI is green on an empty test, and `.env` holds your keys.

Commands

```
# 1. Create the project
mkdir escalate && cd escalate
git init && uv init --python 3.12

# 2. Core dependencies
uv add fastapi uvicorn sqlalchemy alembic pydantic pydantic-settings \
  langgraph langgraph-checkpoint-sqlite openai babel apscheduler jinja2 \
  pytesseract pypdf pillow weasyprint cryptography pyyaml jsonschema \
  google-api-python-client google-auth-oauthlib streamlit httpx

# 3. Dev dependencies
uv add --dev pytest pytest-asyncio freezegun ruff mypy

# 4. Folders
mkdir -p
escalate/{agent,llm,playbooks,scheduler,channels,evidence,export,db,api} \
  ui playbooks/{intl,pk,in,gb,us} templates/{intl,pk,in,gb,us} locales schema
evals tests docs

# 5. Database migrations
uv run alembic init migrations
```

`.env.example` (commit this; never commit `.env`)

```
# LLM: any OpenAI-compatible endpoint. Default = local Ollama ($0, private)
LLM_BASE_URL=http://localhost:11434/v1
LLM_API_KEY=ollama                  # Ollama ignores it; set a real key for
cloud
MODEL_SMART=<local-instruct-model>   # e.g. an open 7-8B instruct model
pulled with `ollama pull`
MODEL_FAST=<local-instruct-model>    # can be the same model

# Free cloud alternatives (uncomment one):
# LLM_BASE_URL=https://api.groq.com/openai/v1
# LLM_BASE_URL=https://generativelanguage.googleapis.com/v1beta/openai/
# LLM_BASE_URL=https://openrouter.ai/api/v1   (use model IDs ending in :free)

CLOUD_PRIVACY_WARNING=true          # warn before real personal data goes to
a cloud free tier
DATABASE_URL=sqlite:///./escalate.db
```

```
EVIDENCE_DIR=./data/evidence
FERNET_KEY=                          # python -c "from cryptography.fernet
import Fernet; print(Fernet.generate_key().decode())"
OCR_LANGS=eng                        # e.g. eng+urd, eng+hin, eng+spa
EMAIL_MODE=smtp                      # smtp (Gmail app password) | gmail_api
SMTP_USER=
SMTP_APP_PASSWORD=
DEFAULT_COUNTRY=                     # ISO code; blank = ask the user
DEMO_MODE=false
```

CI: `.github/workflows/ci.yml`

```
name: ci
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v5
      - run: sudo apt-get install -y tesseract-ocr tesseract-ocr-urd
      - run: uv sync
      - run: uv run ruff check .
      - run: uv run python -m escalate.playbooks.validate playbooks/
      - run: uv run pytest -q
```

Also add on day 0: `LICENSE` (MIT), `.gitignore` (Python + `.env`, `data/`, `secrets/`), and a one-paragraph `README.md` placeholder.

5. Database models

Five tables, written once in `escalate/db/models.py`; Alembic generates the migration with `uv run alembic revision --autogenerate -m "init"`.

```
from datetime import datetime
from enum import StrEnum
from sqlalchemy import ForeignKey, String, Text, JSON, Numeric
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column,
relationship

class Base(DeclarativeBase): ...
```

```
class CaseStatus(StrEnum):
    INTAKE = "intake"; URGENT = "urgent_checklist"; DRAFTING = "drafting"
    AWAITING_APPROVAL = "awaiting_approval"; WAITING_REPLY = "waiting_reply"
    PICK_NEXT = "pick_next"; RESOLVED = "resolved"
    CLOSED_UNRESOLVED = "closed_unresolved"; WITHDRAWN = "withdrawn"

class Case(Base):
    __tablename__ = "cases"
    id: Mapped[int] = mapped_column(primary_key=True)
    case_type: Mapped[str] = mapped_column(String(40))      # refund_refused
| subscription_cancel | scam
    country: Mapped[str] = mapped_column(String(2))           # ISO 3166-1,
any country
    region: Mapped[str | None] = mapped_column(String(10))    # ISO 3166-2,
e.g. US-CA, PK-SD
    language: Mapped[str] = mapped_column(String(5), default="en")  # ISO
639-1 for drafts
    company: Mapped[str | None] = mapped_column(String(200))
    amount: Mapped[float | None] = mapped_column(Numeric(12, 2))
    currency: Mapped[str] = mapped_column(String(3))          # ISO 4217,
from country
    status: Mapped[CaseStatus] = mapped_column(String(30),
default=CaseStatus.INTAKE)
    playbook_id: Mapped[str | None] = mapped_column(String(120))
    playbook_version: Mapped[str | None] = mapped_column(String(20))
    current_step: Mapped[str | None] = mapped_column(String(80))
    next_action_at: Mapped[datetime | None]
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
    facts = relationship("CaseFact", cascade="all, delete-orphan")
    evidence = relationship("Evidence", cascade="all, delete-orphan")
    messages = relationship("Message", cascade="all, delete-orphan")
    events = relationship("Event", cascade="all, delete-orphan")

class CaseFact(Base):
    __tablename__ = "case_facts"
    id: Mapped[int] = mapped_column(primary_key=True)
    case_id: Mapped[int] = mapped_column(ForeignKey("cases.id"))
    key: Mapped[str] = mapped_column(String(80))             # e.g. order_id,
return_shipped_on
    value: Mapped[str] = mapped_column(Text)
    source_evidence_id: Mapped[int | None] =
mapped_column(ForeignKey("evidence.id"))
```

```
    confidence: Mapped[float] = mapped_column(default=1.0)

class Evidence(Base):
    __tablename__ = "evidence"
    id: Mapped[int] = mapped_column(primary_key=True)
    case_id: Mapped[int] = mapped_column(ForeignKey("cases.id"))
    kind: Mapped[str] = mapped_column(String(20))            # image | pdf |
email | text
    file_path: Mapped[str] = mapped_column(String(500))      # encrypted file
    sha256: Mapped[str] = mapped_column(String(64))
    extracted_text: Mapped[str | None] = mapped_column(Text)
    uploaded_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

class Message(Base):
    __tablename__ = "messages"
    id: Mapped[int] = mapped_column(primary_key=True)
    case_id: Mapped[int] = mapped_column(ForeignKey("cases.id"))
    step_id: Mapped[str] = mapped_column(String(80))
    direction: Mapped[str] = mapped_column(String(3))        # out | in
    channel: Mapped[str] = mapped_column(String(20))         # email | post |
court | web_form | info
    recipient: Mapped[str | None] = mapped_column(String(300))
    subject: Mapped[str | None] = mapped_column(String(300))
    body: Mapped[str] = mapped_column(Text)
    status: Mapped[str] = mapped_column(String(20), default="draft")  #
draft|approved|sent|received
    approved_at: Mapped[datetime | None]
    sent_at: Mapped[datetime | None]

class Event(Base):
    __tablename__ = "events"                                  # append-only
audit log
    id: Mapped[int] = mapped_column(primary_key=True)
    case_id: Mapped[int] = mapped_column(ForeignKey("cases.id"))
    type: Mapped[str] = mapped_column(String(40))
    payload: Mapped[dict] = mapped_column(JSON, default=dict)
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
```

6. Playbooks

Playbooks make Escalate global: a generic international ladder works everywhere, and country packs add the exact local route. The Pakistan pack below (seller → platform → 15-day legal notice → District Consumer Court, 30-day filing limit) is the worked example every other country copies.

Files to build

1. `schema/playbook.schema.json` — JSON Schema for every playbook (id, version, country, region, case_type, required_facts, steps, deadlines, sources).
2. `escalate/playbooks/loader.py` — reads YAML, validates, caches by id.
3. `escalate/playbooks/resolver.py` — picks the most specific match: company file → region file → country file.
4. `escalate/playbooks/validate.py` — CLI used in CI; fails on bad schema, missing templates or missing sources.

Global layout

```
playbooks/
├── intl/generic/refund_refused.yaml        # fallback for every country
├── intl/generic/subscription_cancel.yaml
├── intl/generic/scam.yaml
├── pk/ecommerce/refund_refused.yaml        # country packs override intl
├── in/...  gb/...  us/...
└── <cc>/<sector>/<case_type>[.<company>].yaml
templates/<cc>/<lang>/*.md.j2               # e.g. templates/pk/ur/,
templates/in/hi/
```

The generic ladder: **(1)** written demand to the company, 7 days → **(2)** final notice naming the next step, 14 days → **(3)** complaint to the national consumer authority, ombudsman or card issuer (the UI shows a "find your authority" guide and asks the user to paste its address) → **(4)** small-claims or consumer-court guidance. Every generic draft carries the line "Check the rules in your country".

Country packs (`pk` , `in` , `gb` , `us` in v0.1) each list official sources; a `country:<cc>` label and a maintainer per pack keep them current.

First playbook: `playbooks/pk/ecommerce/refund_refused.yaml`

```
id: pk-ecommerce-refund-refused
version: 0.1.0
country: PK
regions: [sindh, punjab, kp, ict]
case_type: refund_refused
last_verified: 2026-10-28
required_facts: [buyer_name, seller_name, seller_contact, order_id,
order_date,
                 amount, defect_description, remedy_wanted]
optional_facts: [return_shipped_on, return_tracking, platform_name,
last_promise_on]
deadlines:
  - id: court_filing_limit
    rule: "30 days from cause of action; court may condone delay with
reasons"
    warn_when_days_left: 10
steps:
  - id: s1_seller_written
    channel: email           # or WhatsApp text the user copies
    template: pk/seller_written_demand.md.j2
    wait_days: 5
    on_no_reply: s2_platform
    on_refused: s3_legal_notice
  - id: s2_platform
    when: "platform_name is set"
    channel: web_form
    template: pk/platform_escalation.md.j2
    wait_days: 5
    on_no_reply: s3_legal_notice
    on_refused: s3_legal_notice
  - id: s3_legal_notice
    channel: post            # registered post or hand delivery with
acknowledgement
    template: pk/legal_notice_15_day.md.j2
    wait_days: 15
    proof_required: delivery_receipt
    on_no_reply: s4_consumer_court
    on_refused: s4_consumer_court
  - id: s4_consumer_court
    channel: court
    template: pk/consumer_court_complaint.md.j2
```

```
    attachments: [timeline_pdf, legal_notice, delivery_receipt, evidence_all]
    adds_if_late: pk/condonation_of_delay.md.j2
sources:
  - title: "CRCP guide: filing a consumer complaint"
    url: "https://crcp.org.pk/wp-content/uploads/2020/10/Complaint-Filing-in-Consumer-courts.pdf"
  - title: "Sindh consumer court filing overview"
    url: "https://commoner-law.com/pakistan/consumer-rights/filing-a-consumer-court-complaint/sindh"
```

Templates for the Pakistan pack (`templates/pk/`)

- `seller_written_demand.md.j2` — polite, firm demand with order facts and a 5-day deadline.
- `platform_escalation.md.j2` — for marketplace support, citing the seller's silence.
- `legal_notice_15_day.md.j2` — formal notice: parties, facts, defect, remedy demanded, 15-day deadline, intent to file under the provincial Consumer Protection Act.
- `consumer_court_complaint.md.j2` — plain-paper complaint: parties, facts, notice proof, relief (replacement/refund + costs).
- `condonation_of_delay.md.j2` — application explaining the delay with dated seller promises.

Have a local lawyer or consumer-rights group review these 5 templates once before launch; record the review in the playbook's changelog.

7. Agent graph

One LangGraph graph per case, checkpointed to SQLite by `thread_id = case_id`, pausing with `interrupt()` at approval and while waiting for a reply.

Nodes

Node	Model slot	Does
`intake`	smart	Extracts facts from text + OCR'd evidence; lists missing required facts
`ask_missing`	—	Pauses for user answers (max 5 questions)

Node	Model slot	Does
`classify`	fast	Case type, country, region, language; resolver picks the playbook
`urgent_checklist`	—	Scam cases only: shows bank/card-block steps first
`draft`	smart	Fills the step template, writes the message in the chosen language
`fact_check`	fast + code	Every amount, date, ID in the draft must exist in facts
`approval`	—	`interrupt()`; resumes only on the user's approve/edit/reject
`send`	—	Sends via channel or marks "user sent it" for post/court
`wait`	—	`interrupt()`; scheduler or a logged reply resumes it
`judge_reply`	fast	resolved / partial / refused / no_reply
`pick_next`	code	Follows `on_no_reply` / `on_refused`; checks deadlines

With one local model, both slots point to it; the code-based fact check keeps quality safe even when the model is small.

Skeleton: `escalate/agent/graph.py`

```
from typing import TypedDict, Literal
from langgraph.graph import StateGraph, END
from langgraph.types import interrupt
from langgraph.checkpoint.sqlite import SqliteSaver

class CaseState(TypedDict, total=False):
    case_id: int
    facts: dict
    missing: list[str]
    playbook_id: str
    step_id: str
    draft: dict              # {recipient, subject, body}
```

```
    decision: Literal["approve", "edit", "reject"]
    reply_verdict: Literal["resolved", "partial", "refused", "no_reply"]

def approval(state: CaseState) -> CaseState:
    answer = interrupt({"type": "approve_draft", "draft": state["draft"]})
    return {"decision": answer["decision"], "draft": answer.get("draft",
state["draft"])}

def wait(state: CaseState) -> CaseState:
    reply = interrupt({"type": "waiting_reply", "step": state["step_id"]})
    return {"last_reply": reply.get("text")}   # None when the timer fired

g = StateGraph(CaseState)
for name, fn in [("intake", intake), ("ask_missing", ask_missing),
("classify", classify),
                 ("urgent_checklist", urgent_checklist), ("draft", draft),
                 ("fact_check", fact_check), ("approval", approval), ("send",
send),
                 ("wait", wait), ("judge_reply", judge_reply), ("pick_next",
pick_next)]:
    g.add_node(name, fn)

g.set_entry_point("intake")
g.add_conditional_edges("intake", lambda s: "ask_missing" if s.get("missing")
else "classify")
g.add_edge("ask_missing", "intake")
g.add_conditional_edges("classify", lambda s: "urgent_checklist" if
s["facts"].get("case_type") == "scam" else "draft")
g.add_edge("urgent_checklist", "draft")
g.add_edge("draft", "fact_check")
g.add_conditional_edges("fact_check", lambda s: "approval" if
s.get("fact_ok") else "draft")
g.add_conditional_edges("approval", lambda s: {"approve": "send", "edit":
"fact_check", "reject": "draft"}[s["decision"]])
g.add_edge("send", "wait")
g.add_edge("wait", "judge_reply")
g.add_conditional_edges("judge_reply", lambda s: END if s["reply_verdict"] ==
"resolved" else "pick_next")
g.add_conditional_edges("pick_next", lambda s: "draft" if s.get("step_id")
else END)

graph =
g.compile(checkpointer=SqliteSaver.from_conn_string("checkpoints.db"))
```

Resume after a user action: `graph.invoke(Command(resume={"decision": "approve"}), config={"configurable": {"thread_id": str(case_id)}})`. The `send` node is the only code that can send, and it refuses unless the DB row has `approved_at` set.

8. Prompts

Five prompts live in `escalate/llm/prompts/` as versioned text files; each returns JSON parsed into a Pydantic model, and every prompt change must pass the evals.

1. Fact extraction (`extract_facts.txt`, smart model)

```
You extract facts for a consumer complaint. Use ONLY the user's text and the
evidence below.
Return JSON: {"facts": {key: {"value": str, "evidence_id": int|null,
"confidence": 0-1}},
              "missing": [required keys you could not find]}
Required keys: {required_facts}
Rules: never guess a value; dates as YYYY-MM-DD; amounts as numbers with
currency;
if two sources disagree, use the evidence and lower confidence.
USER TEXT: {user_text}
EVIDENCE: {evidence_blocks}
```

2. Classification (`classify.txt`, fast model)

```
Classify this complaint. Return JSON: {"case_type":
"refund_refused"|"subscription_cancel"|"scam"|"other",
"country": ISO-3166-1 code|null, "region": ISO-3166-2 code|null, "language":
ISO-639-1 code, "urgent": bool, "reason": str}
urgent=true when money was sent to a likely fraudster in the last 72 hours.
COMPLAINT: {summary}
```

3. Drafting (`draft.txt`, smart model)

```
You write the {step_name} for a consumer, in their own name, to
{recipient_role}.
Start from this template and keep its structure: {template_rendered}
Facts you may use (no others): {facts_json}
Previous messages in this case: {history}
Tone: firm, polite, factual. No threats, insults, exaggeration or legal
claims beyond the template.
Language: {language}. Return JSON: {"subject": str, "body": str}
```

4. Reply judge (`judge_reply.txt`, fast model)

```
A company replied to a consumer's complaint. Decide the outcome.
What the consumer asked for: {remedy_wanted}
THE REPLY: {reply_text}
Return JSON: {"verdict": "resolved"|"partial"|"refused"|"no_reply",
"promise": str|null,
"promise_date": YYYY-MM-DD|null, "reason": str}
A vague promise with no date ("we will replace when the claim is approved")
is "refused".
```

5. Fact check (`fact_check.txt`, fast model + code)

Code first: regex every number, date and ID in the draft and compare with `case_facts`. Then the model checks claims code cannot see:

```
List every factual claim in this draft that is NOT supported by the facts.
DRAFT: {draft}  FACTS: {facts_json}
Return JSON: {"unsupported": [str]}
```

Any unsupported item sends the draft back to `draft` with the list attached.

9. Core services

Five small modules do the non-AI work; each is a plain Python file with its own tests.

Evidence + OCR (`escalate/evidence/`)

1. On upload: compute SHA-256, encrypt with Fernet, save to `EVIDENCE_DIR/<case_id>/<sha>.bin`.

2. Extract text: images → `pytesseract.image_to_data(lang=settings.OCR_LANGS)`; PDFs → `pypdf`; `.eml` → Python `email` module.
3. If mean OCR confidence < 60, optionally send the image to a vision model (local, or a free cloud tier if the user allows); otherwise ask the user to type the key details.
4. Save `extracted_text` and log an `evidence_added` event.

Approval gate (`escalate/channels/gate.py`)

```
class NotApproved(Exception): ...

def require_approval(msg: Message) -> None:
    if msg.status != "approved" or msg.approved_at is None:
        raise NotApproved(f"message {msg.id} not approved")
    if msg.sent_at is not None:
        raise NotApproved(f"message {msg.id} already sent")   # idempotency
```

Every channel's `send()` calls `require_approval()` first; the approve endpoint is the only code that sets `approved_at`.

Email (`escalate/channels/email_gmail.py`)

- OAuth with scope `gmail.send` only; token stored encrypted per user.
- Build MIME with attachments (timeline PDF, evidence) → `users.messages.send`.
- Store the Gmail message id; set `sent_at`; log `message_sent`.
- For `post` and `court` channels: generate a printable PDF and mark sent when the user uploads the courier receipt.

Scheduler (`escalate/scheduler/jobs.py`)

- APScheduler `BackgroundScheduler` with `SQLAlchemyJobStore`; job id `case-<id>-step-<step>` (re-adding replaces, so no duplicates).
- After `send`: schedule `wake_case(case_id)` at `sent_at + wait_days`.
- Daily 9:00 job: scan deadlines (e.g. 30-day court limit) and create "deadline in N days" alerts.
- `wake_case` resumes the graph with `{"text": None}` → judged as `no_reply`.

PDF case file (`escalate/export/case_pdf.py`)

Jinja2 HTML → WeasyPrint. Pages: cover (parties, amount, status) · fact table with sources · dated timeline of every event · full text of each message · evidence thumbnails with SHA-256 hashes.

10. API and UI

The Streamlit UI talks only to the FastAPI backend, so a future WhatsApp bot or Next.js app reuses the same 12 endpoints.

API endpoints

Method	Path	Does
POST	`/cases`	Create a case from free text
POST	`/cases/{id}/evidence`	Upload a file (multipart)
POST	`/cases/{id}/answers`	Answer the agent's missing-fact questions
GET	`/cases`	List cases with status and next action date
GET	`/cases/{id}`	Case detail: facts, current step, pending draft
GET	`/cases/{id}/timeline`	Ordered events
PUT	`/messages/{id}`	Edit a draft
POST	`/messages/{id}/approve`	Approve; triggers send (sets `approved_at`)
POST	`/messages/{id}/reject`	Reject with a reason; agent redrafts
POST	`/cases/{id}/replies`	Log a company reply (paste or forward)
GET	`/cases/{id}/export.pdf`	Download the case file
POST	`/cases/{id}/withdraw`	Stop the case

Streamlit screens (`ui/`)

1. **New case** — big text box ("What happened?"), file uploader, country/province picker.
2. **Questions** — the agent's missing-fact questions as a short form.

3. **Case dashboard** — status badge, next action and date, deadline warnings in red, timeline.
4. **Review draft** — editable subject/body side by side with the facts used; buttons: Approve & send · Edit · Reject.
5. **Log a reply** — paste the company's reply; shows the agent's verdict.
6. **My cases** — table of all cases with status filters.
7. **Settings** — connect Gmail, choose model provider, draft language (any) and UI language (community translations in `locales/`).

11. Testing and evals

Two layers: fast unit tests on every push, and an LLM eval suite on 50 golden cases that gates every prompt or model change.

Unit and integration tests (`tests/`)

- `test_gate.py` — sending without approval raises `NotApproved` from every channel; double-send blocked.
- `test_playbooks.py` — every YAML validates; resolver picks company → region → country correctly.
- `test_scheduler.py` — with `freezegun`, a 15-day wait fires once on day 15, and again after a simulated restart.
- `test_graph.py` — full refund case with a fake LLM and fake mailbox: intake → seller → notice → court draft.
- `test_deadlines.py` — 30-day limit warning appears at 10 days left; condonation template added when late.

Golden-set evals (`evals/`)

Build 50 synthetic cases as JSON (user text + fake evidence + expected facts/outcomes), including the "broken LED, 8 months, waiting for company claim" case.

Eval	Target
Fact extraction field accuracy	≥ 95%
Case type + region correct	≥ 95%

Eval	Target
Unsupported facts in drafts	0
Drafts flagged for threats or rudeness	0
Reply verdict correct (40 sample replies)	≥ 90%

Run with `uv run python -m evals.run --suite all`; it writes `evals/report.md`, committed with each release.

12. Deployment

Users self-host for free with one command on their own computer; the public demo costs nothing because it is a GIF, a video and, optionally, a free-tier hosted copy with sending disabled.

`docker-compose.yml`

```
services:
  ollama:                       # free local LLM; remove if using a cloud key
    image: ollama/ollama
    volumes: ["./data/ollama:/root/.ollama"]
    ports: ["11434:11434"]
  api:
    build: .
    command: uv run uvicorn escalate.api.main:app --host 0.0.0.0 --port 8000
    env_file: .env
    environment: ["LLM_BASE_URL=http://ollama:11434/v1"]
    volumes: ["./data:/app/data", "./secrets:/app/secrets"]
    ports: ["8000:8000"]
    depends_on: [ollama]
  ui:
    build: .
    command: uv run streamlit run ui/app.py --server.port 8501
    environment: ["API_URL=http://api:8000"]
    ports: ["8501:8501"]
    depends_on: [api]
```

First run: `docker compose exec ollama ollama pull <model>`. A `make setup` script does this automatically.

The `Dockerfile` starts from `python:3.12-slim`, installs `tesseract-ocr tesseract-ocr-urd` and WeasyPrint's system libraries, copies the repo and runs `uv sync --frozen`. The scheduler starts inside the API process on startup.

Public demo

- **Always (free):** a 30–45 s demo GIF in the README and a 2-minute YouTube walkthrough of the broken-LED case.
- **Optional hosted demo (free tier):** a Hugging Face Space or Streamlit Community Cloud app using a free cloud LLM key. Free-tier rules change: Hugging Face's free CPU Basic hardware sleeps when idle, and in mid-2026 users reported Docker Spaces being marked as paid for new accounts, so check current terms before choosing.
- Set `DEMO_MODE=true`: no real sending, synthetic sample data only, cases auto-delete after 24 hours, uploads capped at 5 MB.
- Preload the broken-LED sample case so visitors see a full timeline instantly.

13. GitHub launch kit

The README sells the project in 10 seconds: a one-line promise, a demo GIF, and a 3-command quick start.

README outline

1. **Title + one-liner:** "Escalate: the AI agent that fights for your refund, so you don't give up."
2. **Demo GIF** (30–45 s): upload a broken-LED photo → draft appears → approve → timeline jumps to legal notice → PDF case file.
3. **Why it exists:** the 8-month "waiting for company approval" story in 4 sentences.
4. **Quick start:** `git clone`, `cp .env.example .env`, `docker compose up`.
5. **How it works:** the architecture diagram from the design doc.
6. **Safety:** human approval on every message, facts-only drafting, not legal advice.
7. **Playbooks:** supported countries table + "Add your country" link.
8. **Roadmap, Contributing, License (MIT), Acknowledgements.**

Repo files

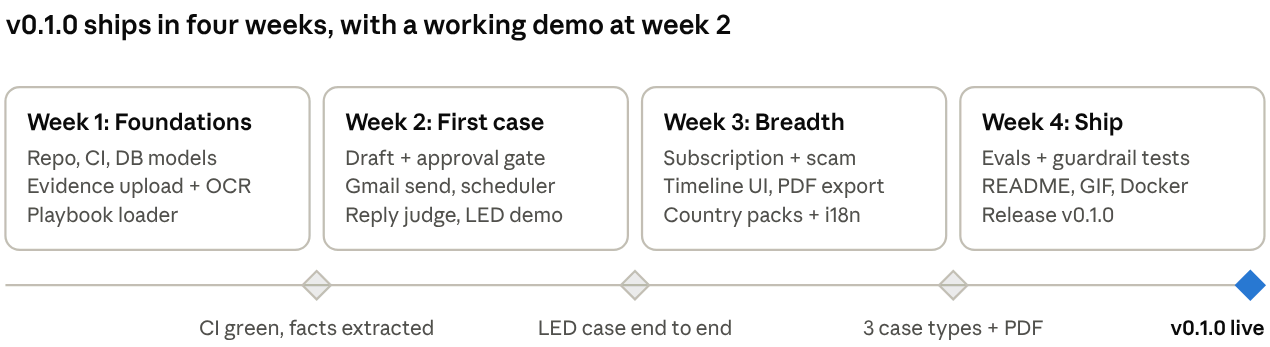
- `CONTRIBUTING.md` and `docs/CONTRIBUTING-PLAYBOOKS.md` (how to write and source a playbook).
- `CODE_OF_CONDUCT.md`, `SECURITY.md` (how to report a vulnerability).
- Issue templates: bug, new playbook request, playbook correction.
- Labels: `good first issue`, `playbook`, `country:pk` — seed 10 good-first issues (e.g. "Add KP playbook", "Urdu translation of templates").
- Topics: `ai-agent`, `consumer-rights`, `langgraph`, `llm`, `legal-tech, i18n, self-hosted`.

Launch posts (same week as v0.1.0)

- LinkedIn: the cousin's LED story → what you built → GIF → link.
- X/Twitter thread: problem stats → demo → "add your country" call to action.
- Reddit: r/SideProject, r/opensource, r/pakistan (follow each sub's self-promotion rules).
- Hacker News "Show HN" and Product Hunt one week later, after fixing first feedback.

14. 4-week schedule

The refund flow works end to end by the end of week 2, so even if later weeks slip there is a demo to show; each week ends at a gate you check before moving on.



4-week roadmap · 4 phases, 4 gates

Week 1 — Foundations

- [ ] Day 0 setup: repo, uv, folders, `.env`, CI green (section 4)

- [ ] DB models + first Alembic migration (section 5)
- [ ] OpenAI-compatible LLM client (Ollama default, free cloud fallback) with Pydantic JSON parsing + retry
- [ ] Evidence upload, encryption, OCR, PDF text (section 9)
- [ ] Playbook schema, loader, resolver, validator CLI (section 6)
- [ ] `intake` + `ask_missing` nodes; extraction prompt; 10 golden cases

Week 2 — First full case

- [ ] `classify`, `draft`, `fact_check`, `approval` nodes + graph wiring (section 7)
- [ ] Approval gate + its tests
- [ ] Gmail SMTP (app password) send; Gmail API as an option
- [ ] Scheduler with job store; `wait` + `judge_reply` + `pick_next`
- [ ] Generic international + Pakistan refund playbooks and templates
- [ ] Minimal Streamlit: new case, review draft, dashboard
- [ ] Run the broken-LED case end to end; record a rough demo

Week 3 — Breadth

- [ ] Subscription-cancel playbook + templates
- [ ] Scam flow: urgent checklist screen, bank and national fraud-report drafts (per country + generic)
- [ ] Consumer court complaint + condonation templates; 30-day deadline tracker
- [ ] Timeline view, log-a-reply screen, My cases, Settings
- [ ] PDF case file export; India, UK and US packs; Babel localisation
- [ ] Grow golden set to 50 cases across at least 5 countries and 3 languages

Week 4 — Ship

- [ ] Full eval run; fix until targets pass (section 11)
- [ ] Template review by a lawyer or consumer-rights group for each country pack
- [ ] Dockerfile + compose; test a fresh install on another machine
- [ ] README, demo GIF, CONTRIBUTING, issue templates, 10 good-first issues (section 13)
- [ ] Public demo in `DEMO_MODE` (section 12)
- [ ] Tag v0.1.0, publish release notes, post launch posts

15. Costs, risks and after v0.1

Building Escalate and running it can cost US$0: a local model, free open-source tools, and the user's own computer as the server. Paid APIs and a domain are optional extras, never requirements.

Costs

Item	Free path	Optional paid
LLM during development + evals	Ollama locally, or free Groq / Gemini / OpenRouter tiers	Paid API for a quality comparison
LLM per user case	User's own local model: $0	User's own paid key, their choice
Hosting	User's own computer via Docker	VPS if someone wants always-on
Demo	GIF + video; free-tier hosted copy if available	—
Email	Gmail SMTP / Gmail API	—
Domain	GitHub Pages URL	~US$10 per year
GitHub, CI, Tesseract, Streamlit, SQLite	Free	—

Risks

Risk	Fix
Legal templates wrong for a country or province	Official sources + `last_verified` in every playbook; one expert review per pack; generic drafts say "check your local rules"
User misses a filing deadline	Deadline tracker warns at 10 days left; late-filing template where the law allows
LLM invents a fact	Code + model fact check blocks approval; 0-tolerance eval

Risk	Fix
Small local model writes weak letters	Templates carry the structure; model only fills facts and tone; eval each recommended model
Free cloud tier changes or disappears	Provider set in `.env`; local Ollama always works offline
Free tiers may train on prompts	Local by default; privacy warning before real data goes to a free cloud tier
Gmail API verification friction	SMTP app-password mode works with no verification
Scope creep	v0.1 = 3 case types, generic + 4 country packs; everything else goes to issues
Burnout as a solo builder	Ship the refund flow first (week 2); the rest is additive

After v0.1

- **v0.2:** Telegram bot (free API), more UI translations; WhatsApp later (its Business API is paid), auto-detect replies from inbox.
- **v0.3:** more countries via community playbooks; scam pack for each bank's fraud desk.
- **v1.0:** Next.js app, multi-user hosting, opt-in anonymous success rate per playbook.